Clase 11: Organización de proyectos, APIs y fechas

Estructura de carpetas, reproducibilidad, buenas prácticas, APIs y lubridate

Nicolás Sidicaro

Ciencia de Datos para Economía y Negocios | FCE-UBA

¿Por qué organizar un proyecto?

El problema

Lo más típico que nos pasa al principio

Situaciones frecuentes

- Tengo 15 versiones de un script: `analisis_v2_final_FINAL2.R`
- No sé qué script hay que correr primero y cuál después.
- Cambié algo y ahora **nada funciona** (y no sé qué cambié).
- Un colega me pide el código y **no le corre** en su computadora.
- Tres meses después, **yo mismo** no entiendo mi propio código.

La buena noticia

Estos problemas se resuelven con hábitos simples de **organización** y **buenas prácticas**. No es difícil: solo hay que conocerlos.

Mi yo del pasado

- Bases
- Epidemio 27-7
- .RData
- .Rhistory
- AWS - Confirmados, fallecidos, vacunados y trabajadores por edad
- Contagios, fallecidos y vacunados por CUIT y edad - Noviembre 2021
- Desciptivos abiertos por edad
- Descriptivos contribuyentes
- Descriptivos fallecidos - Copia de contagios
- Descriptivos vacunación - Sin rangos etarios
- Descriptivos y graficos para informe final - AMBA
- Descriptivos y gráficos para informe final
- Generar proporcion de genero y edad mas 50
- Indice provincias
- Vacunados abiertos por edad

Hoja de ruta de la clase

1. **Estructura de carpetas**: cómo organizar un proyecto de datos.
2. **Scripts encadenados**: modularizar y reproducir el flujo de trabajo.
3. **Buenas prácticas de código**: nombres, estilo, comentarios.
4. **Introducción a APIs**: obtener datos del BCRA y del Banco Mundial.
5. **Lubridate**: manejo profesional de fechas en R.
6. **Bonus**: paquete `janitor` y el problema de las fechas de Excel.

Estructura de carpetas

¿Por qué importa la estructura?

Un proyecto reproducible necesita orden

Si otra persona (o vos en 3 meses) tiene que entender y correr tu análisis, necesita saber:

- ¿Dónde están los **datos crudos**?
- ¿Dónde están los **scripts**?
- ¿Dónde van los **resultados**?
- ¿En qué **orden** se corre todo?

Clave

Los datos crudos son **sagrados**: nunca se modifican. Todo cambio se hace en un script que lee el crudo y genera un archivo procesado. Así evitamos que cambios en los datos afecten los resultados.

Estructura recomendada

```
mi_proyecto/
-- datos/
|   -- crudos/          # Datos originales (NO se tocan)
|   -- procesados/      # Datos limpios generados por scripts
-- scripts/
|   -- 01_descarga.R    # Descarga o copia datos
|   -- 02_limpieza.R    # Limpieza y transformación
|   -- 03_analisis.R    # Modelos, tests, cálculos
|   -- 04_graficos.R    # Visualizaciones
-- resultados/
|   -- tablas/          # Tablas exportadas
|   -- figuras/         # Gráficos exportados
-- utils/
|   -- funcion_x.R      # Funciones personalizadas que se usan en los scripts
-- docs/                # Informes, presentaciones
-- README.md            # Qué es el proyecto, cómo se corre (ver archivo en Github)
-- mi_proyecto.Rproj    # Proyecto de RStudio
```

El archivo README

¿Qué es?

Un archivo de texto (generalmente `.md`) que explica el proyecto en pocas líneas. Es lo primero que lee alguien que encuentra tu carpeta.

Contenido mínimo de un README

1. **Título y descripción**: una oración sobre qué hace el proyecto.
2. **Datos**: de dónde salen y cómo obtenerlos.
3. **Instrucciones**: qué scripts correr y en qué orden.
4. **Dependencias**: qué paquetes hacen falta (`tidyverse`, `lubridate`...).
5. **Autor/a y fecha**.

Para el TP

Su entrega debe incluir un README que permita al docente reproducir todo el trabajo.

Proyectos de RStudio (`.Rproj`)

¿Qué es un `.Rproj`?

Un archivo que RStudio usa para fijar el **directorio de trabajo** en la carpeta del proyecto. Cuando abrís el `.Rproj`, todos los paths son **relativos** a esa carpeta.

Paths absolutos: el error más común

- `read.csv("C:/Users/nico/Desktop/datos/eph.csv")` ← **MAL**
- `read.csv("datos/crudos/eph.csv")` ← **BIEN**

El path absoluto funciona *solo* en tu computadora. El path relativo funciona en cualquiera, siempre que el proyecto esté armado correctamente.

Cómo crear uno

En RStudio: `File > New Project > Existing Directory` y seleccionar la carpeta del proyecto.

Otra opción (la que vimos nosotros)

Armar la carpeta manualmente

Tener una carpeta donde alojamos nuestros trabajos de R para determinada área. Por ejemplo, "C:/Users/nico/Documents/UBA/cdd1c26/". Ahí generar las carpetas mencionadas anteriormente.

Paths absolutos: el error más común

- `read.csv("C:/Users/nico/Documents/UBA/cdd1c26/datos/eph.csv")` ← **MAL**
- `read.csv("datos/eph.csv")` ← **BIEN**

Para que esto funcione sin el Rproj, es necesario definir el wd. Usamos setwd('C:/Users/nico/Documents/UBA/cdd1c26') y hacemos todo relativo a ese path.

Scripts encadenados y reproducibilidad

¿Qué es la reproducibilidad?

Definición práctica

Un análisis es **reproducible** si otra persona puede tomar tus datos y tu código, correrlo, y obtener **exactamente los mismos resultados**.

¿Por qué importa en economía y negocios?

- Un informe económico que no se puede replicar no es creíble.
- Si cambian los datos (nuevo trimestre de EPH), un flujo reproducible se actualiza en minutos.
- Evitamos dolores de cabeza para la actualización.

Referencia

Wilson et al. (2017), "Good Enough Practices in Scientific Computing", resume los principios que seguimos en esta clase.

Scripts encadenados: la idea

Un solo script gigante vs. varios scripts modulares

- Un script de 500 líneas es difícil de navegar, debuggear y mantener.
- Mejor: partir el trabajo en **etapas**, cada una en su propio archivo.

Flujo típico

`01_descarga.R` → `02_limpieza.R` → `03_analisis.R` → `04_graficos.R`

La conexión entre scripts

Cada script **lee** el output del anterior y **genera** un output para el siguiente. El puente son los archivos intermedios guardados en `datos/procesados/`.

Más complejo

Por supuesto, esto se puede complejizar. Veamos el ejemplo 'nsidicarocep/ejemplo_datos_empleo' (link: https://github.com/nsidicarocep/ejemplo_datos_empleo)

Script maestro: `run_all.R`

Un script para correrlos a todos

Opcionalmente, un archivo que ejecuta los demás en secuencia. Así con un solo comando se reproduce el análisis completo.

```r
# run_all.R - Corre el proyecto completo
source("scripts/01_descarga.R")
source("scripts/02_limpieza.R")
source("scripts/03_analisis.R")
source("scripts/04_graficos.R")
```

Condición para que funcione

Cada script debe ser **autocontenido**: cargar sus propios paquetes, leer sus propios inputs y no depender de objetos en memoria de otro script.

Formato de archivos intermedios

- `.rds` para objetos de R (preserva tipos, es rápido).
- `.csv` si querés portabilidad fuera de R.

Buenas prácticas de código

Nombres de objetos y archivos

Reglas para nombrar objetos en R

- Usar `snake_case`: `ingreso_horario`, `tasa_empleo`.
- Nombres **descriptivos**: `df_limpia` dice más que `d2`.
- Evitar nombres de funciones existentes: no llamar `data`, `mean`, `filter`.

Reglas para archivos

- Nombres en minúscula, sin espacios ni acentos.
- Usar guion bajo: `01_limpieza.R`, no `01-limpieza.R` ni `01 Limpieza.R`.
- Numerar si hay orden: `01_`, `02_`, `03_`...

¿Y las columnas de un dataframe?

El paquete `janitor` tiene `clean_names()`: convierte cualquier nombre raro a `snake_case`. Lo vemos al final.

Estilo y legibilidad

Espacios y formato

- Espacio después de comas y alrededor de <- y operadores.
- Una operación por línea en pipes largos.
- Líneas de como máximo 80 caracteres (RStudio lo marca).

```r
# MAL
resultado<-datos|>filter(anio==2024)|>group_by(prov)|>summarise(m=mean(ing))

# BIEN
resultado <- datos |>
  filter(anio == 2024) |>
  group_by(prov) |>
  summarise(ingreso_medio = mean(ingreso, na.rm = TRUE))
```

Guía de estilo

Hay varias: *Tidyverse Style Guide* (`style.tidyverse.org`) es la más usada. El paquete `styler` reformatea código automáticamente.

Comentarios: cuándo y cómo

Regla práctica

Un buen comentario explica **por qué**, no **qué**. El código bien escrito ya dice qué hace.

```
# MAL: repite lo que ya dice el código
# Filtro por estado igual a 1
datos <- datos |> filter(ESTADO == 1)

# BIEN: explica el criterio sustantivo
# Nos quedamos solo con los ocupados (ESTADO == 1, según manual EPH)
datos <- datos |> filter(ESTADO == 1)
```

Secciones

Usar comentarios largos como separadores de secciones:

```
# --- Limpieza de variables de ingreso ---
```

Errores frecuentes a evitar

Cosas que rompen la reproducibilidad

1. **Paths absolutos**: ya lo vimos. Usar `.Rproj` y paths relativos.
2. **Instalar paquetes dentro del script**: `install.packages()` va en la consola, no en el script.
3. **Depender del workspace**: no guardar/cargar `.RData`. Cada script debe ser autocontenido.
4. **No declarar los paquetes**: siempre usar `library()` al inicio.

Configuración recomendada en RStudio

`Tools > Global Options`: desmarcar "Restore .RData into workspace" y poner "Save workspace" en **Never**. Así cada sesión arranca limpia.

Introducción a APIs

¿Qué es una API?

Application Programming Interface

Es una forma de **pedir datos a un servidor** usando una URL estructurada. En lugar de descargar un CSV a mano, le pedimos al servidor que nos devuelva los datos directamente a R.

¿Por qué usar APIs?

Ventajas sobre la descarga manual

- **Automatizable**: un script puede actualizar los datos sin intervención humana.
- **Reproducible**: el pedido queda documentado en el código.
- **Actualizado**: siempre obtenés el dato más reciente.
- **Eficiente**: pedís solo lo que necesitás, no un archivo gigante.

¿Y las desventajas?

- Necesitás conexión a internet.
- Si la API cambia o se cae, tu script se rompe (por eso guardamos una copia local).
- Algunas APIs requieren registro y **API key** (clave de acceso).

Anatomía de una llamada a una API REST

Los componentes de un pedido

1. **Base URL**: la dirección del servidor. Ej: `https://api.bcra.gob.ar`
2. **Endpoint**: qué recurso pedís. Ej: `/estadisticas/v2.0/DatosVariable/1/2024-01-01/2024-12-31`
3. **Método**: casi siempre `GET` ("traeme datos").
4. **Respuesta**: un objeto **JSON** que R convierte a lista o dataframe.

¿Qué es JSON?

JavaScript Object Notation: un formato de texto con pares clave-valor. Es el "CSV de las APIs": liviano, legible y universal.

```
{"fecha": "2024-01-02", "valor": 813.25}
```

Ejemplo 1: API del BCRA — Tipo de cambio oficial

```r
library(httr)
library(jsonlite)
# API de Estadísticas Cambiarias v1.0
# Endpoint: evolución del dólar (USD) en un rango de fechas
url_bcra <- paste0(
  "https://api.bcra.gob.ar/estadisticascambiarias/v1.0/",
  "Cotizaciones/USD",
  "?fechaDesde=2026-04-01&fechaHasta=2026-04-25"
)
respuesta <- GET(url_bcra)
contenido <- content(respuesta, as = "text", encoding = "UTF-8")
datos_bcra <- fromJSON(contenido)
# Los datos vienen en $results (lista de fechas con detalle)
tc_usd <- datos_bcra$results |> unnest(detalle) |> as_tibble() |>
  mutate(fecha = as.Date(fecha))
```

La API del BCRA 2

Endpoints disponibles

- `/Maestros/Divisas`: listado de monedas ISO disponibles.
- `/Cotizaciones?fecha=YYYY-MM-DD`: todas las cotizaciones de una fecha.
- `/Cotizaciones/{codMoneda}?fechaDesde=... &fechaHasta=...`: serie histórica de una moneda.

Atención

La API no requiere API key, pero tiene **límite de consultas por minuto**. Los parámetros `limit` y `offset` permiten paginar resultados largos.

Ejemplo 2: API del Banco Mundial con `WDI`

El paquete WDI

`WDI` es un wrapper de R para la API del Banco Mundial. Simplifica enormemente la consulta: no hay que armar URLs ni parsear JSON.

```r
library(WDI)

# PBI per cápita (US$ constantes 2015) para Argentina, 2000-2023
pbi_arg <- WDI(
  country   = "AR",
  indicator = c("pbi_pc" = "NY.GDP.PCAP.KD"),
  start     = 2000,
  end       = 2023
)

pbi_arg |>
  ggplot(aes(x = year, y = pbi_pc)) +
  geom_line(color = "steelblue", linewidth = 1) +
  labs(title = "PBI per cápita de Argentina (US$ constantes 2015)",
```

¿Cómo encuentro los indicadores del Banco Mundial?

```r
# Buscar indicadores por palabra clave
WDIsearch("inflation") |> head(10)

# Indicadores útiles para economistas:
# NY.GDP.PCAP.KD     -> PBI per cápita (US$ constantes)
# NY.GDP.MKTP.KD.ZG  -> Crecimiento del PBI (% anual)
# FP.CPI.TOTL.ZG     -> Inflación (IPC, % anual)
# SL.UEM.TOTL.ZS     -> Desempleo (% de fuerza laboral)
# NE.EXP.GNFS.ZS     -> Exportaciones (% del PBI)
# SI.POV.GINI        -> Índice de Gini
```

Comparar países

Podés pedir varios países: `country = c("AR", "BR", "CL", "UY")`. Esto es muy útil para análisis comparados.

Buena práctica con APIs

Siempre guardar una **copia local** de los datos descargados. Así el análisis funciona aunque la API se caiga. Lo vimos en el script `01_descarga.R`.

Otras APIs útiles para economistas

APIs de acceso público (Argentina)

- **BCRA** (`api.bcra.gob.ar`): tipo de cambio, tasas de interés, reservas, base monetaria.
- **Ministerio de Economía — series**: `apis.datos.gob.ar/series` — más de 2.000 series económicas.
- **INDEC**: no tiene API pública, pero se puede scrapear (lo veremos al final del curso).

APIs internacionales

- **Banco Mundial** (`WDI`): cientos de indicadores para todos los países.
- **FRED** (Federal Reserve): series de EE.UU. y globales (paquete `fredr`).
- **OECD**: datos de países OCDE (paquete `OECD`).
- **Yahoo Finance** (paquete `quantmod`): precios de acciones y commodities.

APIs de modelos de inteligencia artificial

¿Qué son?

Servicios como OpenAI (GPT), Anthropic (Claude) o Google (Gemini) exponen sus modelos de lenguaje a través de APIs. Enviás un pedido (un *prompt*) y recibís una respuesta en JSON (pagos).

¿Para qué le sirven a un economista?

- **Clasificar texto a escala**: tomar miles de noticias económicas y clasificarlas como "positiva", "negativa" o "neutral" (análisis de sentimiento).
- **Extraer datos de texto no estructurado**: leer actas del BCRA, balances o informes de organismos y extraer cifras, fechas o entidades mencionadas.
- **Estandarizar y limpiar datos**: homogeneizar nombres de empresas, sectores o productos que vienen escritos de formas distintas en cada fuente.

Clave

La API es lo que permite **automatizar** estas tareas para miles de observaciones. Copiar y

Lubridate: manejo de fechas

¿Por qué las fechas son difíciles?

Las fechas parecen simples. . . hasta que no lo son.

Problemas reales con fechas

- ¿01/02/2024 es 1 de febrero o 2 de enero? (depende del país)
- ¿Cuántos días tiene febrero? (depende del año)
- ¿Un mes tiene 30 días? (depende del mes)
- ¿Cuántas semanas hay en un año? (52.14, no 52)

Moraleja

Nunca tratar las fechas como texto ni como números sueltos. R (y todos los lenguajes) tienen tipos de datos específicos para fechas.

Cómo almacenan las fechas las computadoras

El estándar: días desde una fecha de referencia

Internamente, R guarda las fechas como un **número entero**: la cantidad de días desde el 1 de enero de 1970 (el "epoch" de Unix).

```r
hoy <- as.Date("2026-04-28")
as.numeric(hoy)   # Días desde 1970-01-01
```

```
[1] 20571
```

¿Por qué esto importa?

- Permite hacer aritmética: `hoy + 30` da la fecha dentro de 30 días.
- Permite comparar: `fecha1 > fecha2`.
- Permite ordenar cronológicamente.

El formato ISO 8601

El estándar internacional para escribir fechas es `AAAA-MM-DD` (año-mes-día). Es el formato que R usa por defecto con `as.Date()`: no hay ambigüedad.

Tipos de datos temporales en R

Tres tipos

- `Date`: solo fecha (`2026-04-28`). Internamente, un entero (días desde 1970).
- `POSIXct`: fecha + hora (`2026-04-28 14:30:00`). Internamente, segundos desde 1970.
- `Period/Duration/Interval`: lapsos de tiempo (los maneja `lubridate`).

¿Cuándo uso cada uno?

- **Datos económicos** (PBI trimestral, IPC mensual, cotizaciones diarias): `Date` alcanza.
- **Datos financieros intradiarios** (operaciones bursátiles por segundo): `POSIXct`.
- La mayoría de las veces en esta materia usaremos `Date`.

Lubridate: la navaja suiza de las fechas

¿Qué es lubridate?

Un paquete del `tidyverse` que simplifica la creación, manipulación y aritmética de fechas. Se carga con `library(lubridate)`.

Funciones para parsear fechas

`lubridate` tiene funciones que se nombran según el **orden** de los componentes:

- `ymd("2026-04-28")` → año-mes-día
- `dmy("28/04/2026")` → día-mes-año (formato argentino)
- `mdy("04-28-2026")` → mes-día-año (formato estadounidense)
- `dmy_hms("28/04/2026 14:30:00")` → con hora, minuto, segundo

La ventaja

No necesitás memorizar códigos de formato como `%d/%m/%Y`. Solo indicás el orden (dmy, ymd, mdy) y `lubridate` se encarga del resto.

Parseo de fechas: ejemplos

```r
library(lubridate)

# Distintos formatos, misma fecha
ymd("2026-04-28")
```

```
[1] "2026-04-28"
```

```r
dmy("28/04/2026")
```

```
[1] "2026-04-28"
```

```r
dmy("28-abril-2026")
```

```
[1] "2026-04-28"
```

```r
mdy("April 28, 2026")
```

```
[1] "2026-04-28"
```

Parseo de fechas: ejemplos

Cuando el formato es ambiguo

`"01/02/2026"`: ¿es 1 de febrero o 2 de enero?

- `dmy("01/02/2026")` $\rightarrow$ 1 de febrero (formato AR/EU).
- `mdy("01/02/2026")` $\rightarrow$ 2 de enero (formato US).

Hay que saber el formato de tus datos y usar la función correcta.

Extraer componentes de una fecha

```r
fecha <- ymd("2026-04-28")

year(fecha)     # Año
```

```
[1] 2026
```

```r
month(fecha)    # Mes (numérico)
```

```
[1] 4
```

```r
quarter(fecha)  # Trimestre
```

```
[1] 2
```

```r
day(fecha)      # Día del mes
```

```
[1] 28
```

```r
wday(fecha, label = TRUE)   # Día de la semana
```

```
[1] mar\\.
Levels: dom\\. < lun\\. < mar\\. < mié\\. < jue\\. < vie\\. < sáb\\.
```

Extraer componentes de una fecha

¿Para qué sirve?

Para agrupar datos por período: `mutate(anio = year(fecha), trimestre = quarter(fecha))` y luego `group_by(anio, trimestre)`.

Aritmética con fechas

```r
inicio <- ymd("2026-03-13")  # Inicio del curso
fin    <- ymd("2026-07-03")  # Fin del curso

# Diferencia en días
fin - inicio
```

```
Time difference of 112 days
```

```r
# Sumar períodos
inicio + months(3)
```

```
[1] "2026-06-13"
```

```r
inicio + weeks(2) + days(3)
```

```
[1] "2026-03-30"
```

Periods vs Durations

- `months(1)`: un mes **calendario** (28, 29, 30 o 31 días según el mes).
- `dmonths(1)`: exactamente 30.44 días (duración promedio).

Para datos económicos trimestrales o mensuales, usar **periods** (`months()`).

Operaciones frecuentes con fechas económicas

```r
# Redondear fechas: útil para agregar a trimestre
fecha <- ymd("2026-05-15")
floor_date(fecha, unit = "quarter")  # Inicio del trimestre
```

```
[1] "2026-04-01"
```

```r
ceiling_date(fecha, unit = "month")  # Inicio del mes siguiente
```

```
[1] "2026-06-01"
```

```r
# Generar secuencia de fechas
fechas_mensuales <- seq(ymd("2025-01-01"), ymd("2025-12-01"),
                        by = "month")
fechas_mensuales
```

```
 [1] "2025-01-01" "2025-02-01" "2025-03-01" "2025-04-01" "2025-05-01"
 [6] "2025-06-01" "2025-07-01" "2025-08-01" "2025-09-01" "2025-10-01"
[11] "2025-11-01" "2025-12-01"
```

Operaciones frecuentes con fechas económicas

Uso con datos reales

Si tenés un dataframe con fecha, podés crear columnas de período: `mutate(trimestre = floor_date(fecha, "quarter"))` y luego agregar (`group_by + summarise`).

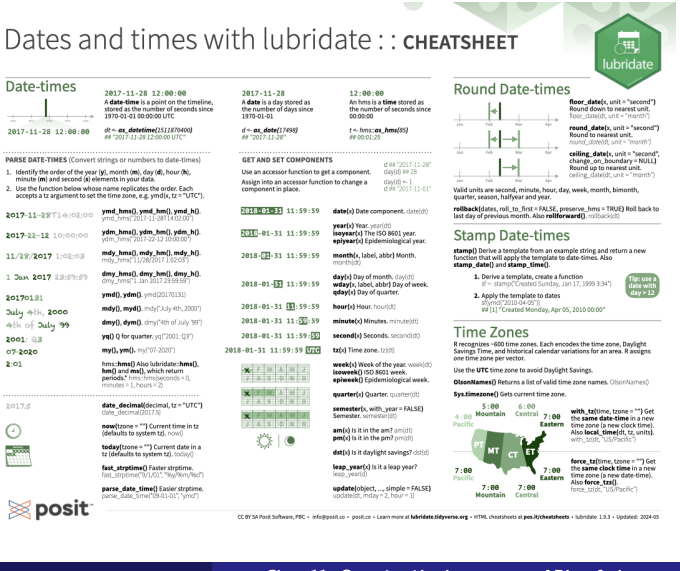
Ejemplo integrador: serie del BCRA con fechas

```r
# Supongamos que descargamos tipo de cambio del BCRA
# y la columna fecha viene como texto "dd/mm/yyyy"

tc <- tc_oficial |>
  mutate(
    fecha     = dmy(fecha),           # Parsear
    anio      = year(fecha),          # Extraer año
    mes       = month(fecha),         # Extraer mes
    trimestre = quarter(fecha),       # Extraer trimestre
    dia_semana = wday(fecha, label = TRUE)
  )

# Promedio mensual del tipo de cambio
tc_mensual <- tc |>
  mutate(mes_inicio = floor_date(fecha, "month")) |>
  group_by(mes_inicio) |>
  summarise(tc_promedio = mean(valor, na.rm = TRUE))
```

Por suerte, tiene cheatsheets (link en cronograma)



Dates and times with lubridate : : CHEATSHEET

lubridate

Date-times

2017-11-28 12:00:00

2017-11-28 12:00:00

A **date-time** is a point on the timeline, stored as the number of seconds since 1970-01-01 00:00:00 UTC

dt <- as_datetime(1511870400)
"2017-11-28 12:00:00 UTC"

2017-11-28

A **date** is a day stored as the number of days since 1970-01-01

d <- as_date(17498)
"2017-11-28"

12:00:00

An hms is a **time** stored as the number of seconds since 00:00:00

t <- hms::as_hms(85)
00:01:25

PARSE DATE-TIMES (Convert strings or numbers to date-times)

1. Identify the order of the year (**y**), month (**m**), day (**d**), hour (**h**), minute (**m**) and second (**s**) elements in your data.
2. Use the function below whose name replicates the order. Each accepts a tz argument to set the time zone, e.g. ymd(x, tz = "UTC").

2017-11-28T14:02:00 — **ymd_hms()**, **ymd_hm()**, **ymd_h()**. ymd_hms("2017-11-28T14:02:00")

2017-22-12 10:00:00 — **ydm_hms()**, **ydm_hm()**, **ydm_h()**. ydm_hms("2017-22-12 10:00:00")

11/28/2017 1:02:03 — **mdy_hms()**, **mdy_hm()**, **mdy_h()**. mdy_hms("11/28/2017 1:02:03")

1 Jan 2017 23:59:59 — **dmy_hms()**, **dmy_hm()**, **dmy_h()**. dmy_hms("1 Jan 2017 23:59:59")

20170131 — **ymd()**, **ydm()**. ymd(20170131)

July 4th, 2000 — **mdy()**, **myd()**. mdy("July 4th, 2000")

4th of July '99 — **dmy()**, **dym()**. dmy("4th of July '99")

2001: Q3 — **yq()** Q for quarter. yq("2001: Q3")

07-2020 — **my()**, **ym()**. my("07-2020")

2:01 — hms::**hms()** Also lubridate::**hm()**, **ms()** and **ms()**, which return **periods**.* hms::hms(seconds = 0, minutes = 1, hours = 2)

2017.5 — **date_decimal**(decimal, tz = "UTC") date_decimal(2017.5)

now(tzone = "") Current time in tz (**defaults to system tz**). now()

today(tzone = "") Current date in a tz (**defaults to system tz**). today()

fast_strptime() Faster strptime. fast_strptime("9/1/01", "%y/%m/%d")

parse_date_time() Easier strptime. parse_date_time("09-01-01", "ymd")

GET AND SET COMPONENTS

Use an accessor function to get a component.

Assign into an accessor function to change a component in place.

d ## "2017-11-28"
day(d) ## 28
day(d) <- 1
d ## "2017-11-01"

2018-01-31 11:59:59

date(x) Date component. date(dt)

2018-01-31 11:59:59

year(x) Year. year(dt)
isoyear(x) The ISO 8601 year.
epiyear(x) Epidemiological year.

2018-01-31 11:59:59

month(x, label, abbr) Month. month(dt)

2018-01-31 11:59:59

day(x) Day of month. day(dt)
wday(x, label, abbr) Day of week.
qday(x) Day of quarter.

2018-01-31 11:59:59

hour(x) Hour. hour(dt)

2018-01-31 11:59:59

minute(x) Minutes. minute(dt)

2018-01-31 11:59:59

second(x) Seconds. second(dt)

2018-01-31 11:59:59 UTC

tz(x) Time zone. tz(dt)

week(x) Week of the year. week(dt)
isoweek() ISO 8601 week.
epiweek() Epidemiological week.

quarter(x) Quarter. quarter(dt)

semester(x, with_year = FALSE) Semester. semester(dt)

am(x) Is it in the am? am(dt)
pm(x) Is it in the pm? pm(dt)

dst(x) Is it daylight savings? dst(dt)

leap_year(x) Is it a leap year? leap_year(dt)

update(object, ..., simple = FALSE) update(dt, mday = 2, hour = 1)

Round Date-times

floor_date(x) — round_date(x) — ceiling_date(x)

floor_date(x, unit = "second") Round down to nearest unit. floor_date(dt, unit = "month")

round_date(x, unit = "second") Round to nearest unit. round_date(dt, unit = "month")

ceiling_date(x, unit = "second", change_on_boundary = NULL) Round up to nearest unit. ceiling_date(dt, unit = "month")

Valid units are second, minute, hour, day, week, month, bimonth, quarter, season, halfyear and year.

rollback(dates, roll_to_first = FALSE, preserve_hms = TRUE) Roll back to last day of previous month. Also **rollforward()**. rollback(dt)

Stamp Date-times

stamp() Derive a template from an example string and return a new function that will apply the template to date-times. Also **stamp_date()** and **stamp_time()**.

1. Derive a template, create a function
   d <- stamp("Created Sunday, Jan 17, 1999 3:34")
   #1 [1] "Created Monday, Apr 05, 2010 00:00"
2. Apply the template to dates
   d(ymd("2010-04-05"))

Tip: use a date with day > 12

Time Zones

R recognizes ~600 time zones. Each encodes the time zone, Daylight Savings Time, and historical calendar variations for an area. R assigns one time zone per vector.

Use the **UTC** time zone to avoid Daylight Savings.

OlsonNames() Returns a list of valid time zone names. OlsonNames()

Sys.timezone() Gets current time zone.

with_tz(time, tzone = "") Get the **same date-time** in a new time zone (a new clock time). Also **local_time**(dt, tz, units). with_tz(dt, "US/Pacific")

force_tz(time, tzone = "") Get the **same clock time** in a new time zone (a new date-time). Also **force_tzs()**. force_tz(dt, "US/Pacific")

posit

CC BY SA Posit Software, PBC • info@posit.co • posit.co • Learn more at **lubridate.tidyverse.org** • HTML cheatsheets at **posit.it/cheatsheets** • lubridate 1.9.3 • Updated: 2024-05

Bonus: janitor y las fechas de Excel

El paquete janitor

¿Qué hace janitor?

Es un paquete de limpieza de datos. La función estrella es `clean_names()`: toma un dataframe y convierte todos los nombres de columna a `snake_case` limpio.

```r
library(janitor)

# df con nombres feos
df <- tibble(
  `Nombre Completo` = "Ana",
  `INGRESO (AR$)` = 500000,
  `% Variación` = 3.2,
  `Fecha.de" Nacimiento` = "1990-05-12"
)

df |> clean_names() |> names()
```

```
[1] "nombre_completo"     "ingreso_ar"          "percent_variacion"
[4] "fecha_de_nacimiento"
```

El paquete janitor

Otras funciones útiles de janitor

- `tabyl()`: tablas de frecuencia más útiles que `table()`.
- `remove_empty()`: elimina filas y columnas vacías.
- `get_dupes()`: detecta filas duplicadas.

El problema de las fechas de Excel

¿Por qué Excel arruina las fechas?

Excel guarda las fechas como **números enteros**: la cantidad de días desde el 1 de enero de 1900 (¡no desde 1970 como R!). Pero además:

- Excel cree erróneamente que 1900 fue bisiesto (bug heredado de Lotus 1-2-3).
- Cuando importás un CSV que Excel tocó, las fechas pueden llegar como números: 44927 en vez de "2023-01-01".
- A veces Excel convierte *cosas que no son fechas* en fechas. Hay un caso famoso de un gen (SEPTIN-1) al que le tuvieron que cambiar el nombre para que Excel no lo rompa (https://en.wikipedia.org/wiki/SEPTIN1)

Cómo convertir las “fechas” de Excel

El problema concreto

Abrir un CSV y en lugar de "15/03/2024" aparece **45366**. ¿Cómo se lleva a fecha?

```r
# janitor tiene la función excel_numeric_to_date()
library(janitor)

# Convertir número de Excel a fecha de R
excel_numeric_to_date(45366)
```

```
[1] "2024-03-15"
```

```r
# También funciona con vectores
numeros_excel <- c(44927, 45000, 45366)
excel_numeric_to_date(numeros_excel)
```

```
[1] "2023-01-01" "2023-03-15" "2024-03-15"
```

Cómo convertir las “fechas” de Excel

Alternativa

También se puede hacer manualmente: el epoch de Excel es "1899-12-30" (por el bug del bisiesto).

```r
as.Date(45366, origin = "1899-12-30")
```

```
[1] "2024-03-15"
```

Bibliografía y recursos

Lecturas de la clase

- Wickham, Çetinkaya-Rundel & Grolemund (2023). *R for Data Science*, 2e. Cap. 6 (Workflow: scripts and projects) y Cap. 17 (Dates and times). `r4ds.hadley.nz`
- Wilson et al. (2017). "Good Enough Practices in Scientific Computing". *PLoS Computational Biology*, 13(6).

Documentación

- `lubridate`: `lubridate.tidyverse.org`
- `janitor`: `sfirke.github.io/janitor`
- API BCRA: `api.bcra.gob.ar`
- WDI: `cran.r-project.org/package=WDI`